



PROGRAM 1 ASSIGNMENT INSTRUCTIONS

CSC 1310, FALL 2026

Movie Theater Management System

Program Description

You have been hired to create a small **Movie Theater Management System** for a local cinema. The program will keep track of the movies currently available at the theater and the individual showings scheduled for those movies.

You will write (almost) the entire program from scratch. Only the Driver.cpp file with the main function is provided. Your finished program will use three cooperating classes—**Movie**, **Showing**, **Theater**—plus a Driver source file. The program must dynamically allocate its arrays and objects and correctly connect each Showing to its associated Movie through a pointer.

Learning Goals

- Design and implement multiple C++ classes using separate specification (.h) and implementation (.cpp) files.
- Create and manage dynamically allocated arrays of pointers.
- Use pointers to connect a Showing object to an existing Movie object without copying the Movie.
- Practice constructors, destructors, accessors, mutators, member functions, input validation, and menu-driven program design.
- Compile a multi-file C++ program and release all dynamically allocated memory.

Submission

- Submit one .zip file named **[YOUR_TTU_USERNAME]_prog1.zip** to Canvas **Program 1** Assignment under Module 1.
- Your zip file must contain **Driver.cpp**, **Movie.h**, **Movie.cpp**, **Showing.h**, **Showing.cpp**, **Theater.h**, **Theater.cpp**, and **Prog1Report**.

Important Rules / Program Constraints

Policy Concerning What You Can't Use in Programming Assignments

You may not use code or syntax in your programming & lab assignments that have not been taught in the class without referencing your source. **You may not ever use the following in your programs:**

- Defining variables throughout a function instead of at the top of the function.
- Defining entire functions (including mutators and accessors) on a single line
- Placing entire if statement including statement attached to if on a single line
- while(true) or while(1)
Check out <https://sites.tntech.edu/acrockett/2025/08/15/why-is-whiletrue-or-while1-bad-programming-practice/> to see why this is bad programming practice.
- goto statements
Check out <https://sites.tntech.edu/acrockett/2025/08/15/why-is-goto-bad-programming-practice-in-c/> to see why this is bad programming practice.
- recursively calling main
- exit()
- ternary operators (any syntax requiring a '?')
- range-based for loops
- try-catch
- getline(cin>>ws, variable);
- lambda functions
- defining all programmer-defined functions above the main function
- code that has no comment block at the top indicating the filename, author, date, and purpose

If you use any of the things listed above or use code that has not been taught in the class (without referencing your detailed source in a comment) then full points will be deducted for that criteria item in the rubric.

Generative AI Policy for This Assignment

- You may only use generative AI to troubleshoot issues.
- You may not upload this assignment to any Generative AI tool.
- You may not use any generated code in your submission.
- You may copy a syntax error to generative AI to ask what it means and ideas on how to fix it. If you do use generative AI to assist in troubleshooting in this way, you must provide a link to your GenAI conversation or if a link is not possible, you must create an additional document providing screenshots and descriptions on how you used Generative AI.

Other requirements of this Programming Assignment

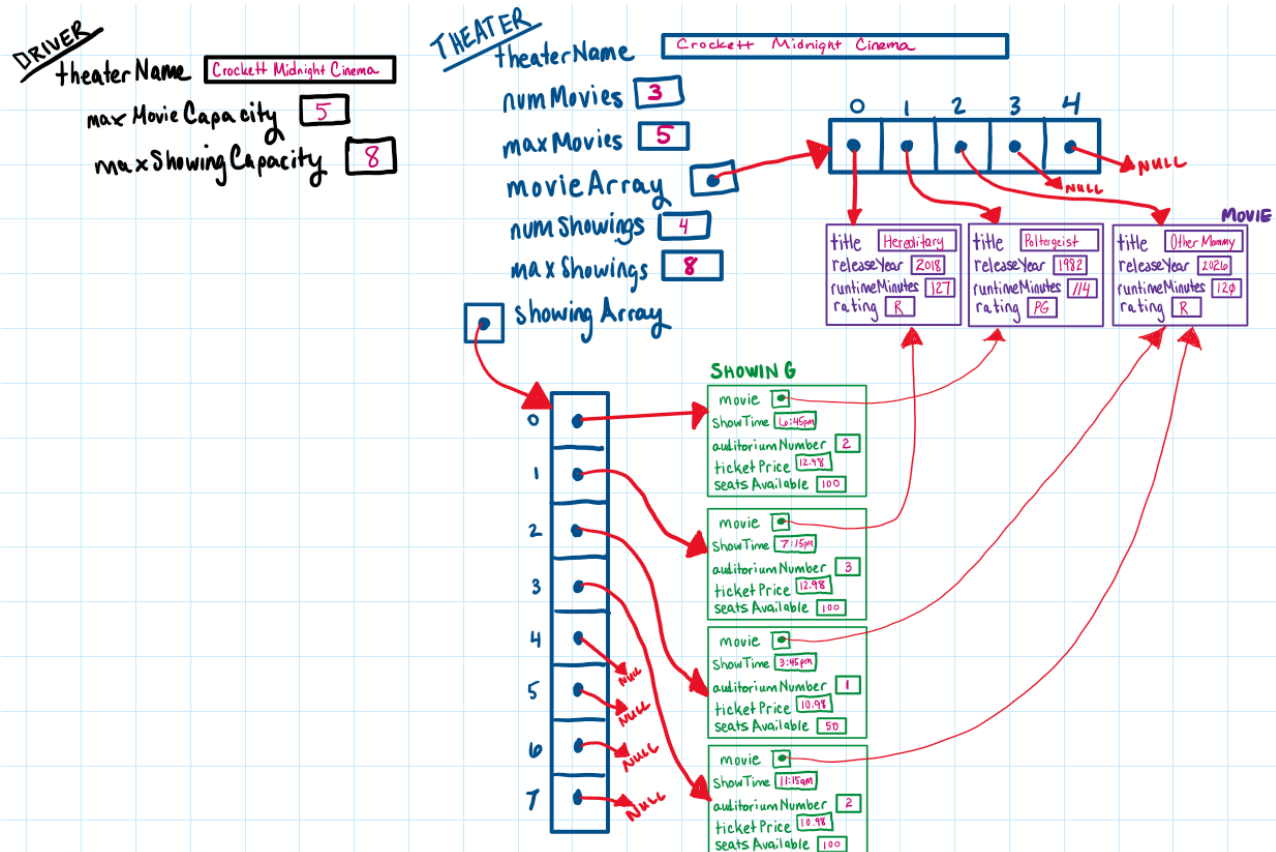
You are expected to build the program using the material and programming practices taught in CSC 1310. In addition to the normal course rules for programming assignments, this assignment has the following structural requirements:

- All three classes must use private data members and public member functions.
- Each class must be split into a header file and a source file.
- The Theater class must dynamically allocate an array of Movie pointers and an array of Showing pointers.
- Each Movie added to the theater must be dynamically allocated.
- Each Showing added to the theater must be dynamically allocated.
- A Showing must store a Movie* that points to an existing Movie object in the Theater movie array.
- The capacities chosen when the Theater is created are fixed. Arrays do not resize in this assignment.
- The Theater destructor must release all dynamically allocated Showing objects, Movie objects, and both dynamically allocated arrays.
- A Showing does not own its Movie object and therefore must not delete the Movie pointer it stores.
- Do not use global variables.

Reminder: The capacities represent how many Movie* and Showing* elements the Theater arrays can hold—not how many objects have currently been added.

Program Requirements / Data Organization

The diagram below shows the required relationship among the objects. Your variable names do not have to match the handwritten labels exactly, but your program must use the same data relationships: the Theater owns arrays of pointers, and each Showing points to one Movie already stored by the Theater.



Required Files

File	Purpose
Driver.cpp	Provided for you; contains main()
Movie.h	Class specification for Movie.
Movie.cpp	Movie member-function definitions.
Showing.h	Class specification for Showing.
Showing.cpp	Showing member-function definitions.
Theater.h	Class specification for Theater.
Theater.cpp	Theater member-function definitions.
Prog1Report	Screenshot of proof (confirmation page) of filling out out Program 1 report.

Movie Class

A Movie object stores the descriptive information for one movie. It does not know anything about the Theater or about when the movie is being shown.

Required private data members:

title	string	Title of the movie.
releaseYear	int	Year the movie was released.
runtimeMinutes	int	Length of the movie in minutes.
rating	string	Movie rating, such as G, PG, PG-13, R, or another appropriate value.

Required public member functions:

Function	Returns	Required behavior / purpose
Movie()	—	Default constructor. Initialize the title to an empty string, numeric values to 0, and the rating to "Unknown".
Movie(...)	—	Parameterized (overloaded) constructor. Accept values for all four Movie attributes and initialize the object.
setTitle	void	Mutator that changes the title.
setReleaseYear	void	Mutator that changes the release year.
setRuntimeMinutes	void	Mutator that changes the runtime in minutes.
setRating	void	Mutator that changes the rating.
getTitle	string	Return the title.
getReleaseYear	int	Return the release year.
getRuntimeMinutes	int	Return the runtime in minutes.
getRating	string	Return the rating.
editMovie	void	Display an edit menu that allows the user to repeatedly change the title, release year, runtime, or rating until the user chooses Done Editing.
printMovie	void	Print all Movie details in an easy-to-read format. If release year or runtime is 0, print Unknown for that value.

Showing Class

A Showing object represents one scheduled screening. Multiple Showing objects may point to the same Movie object. The Showing class stores a pointer to the Movie; it does not make or own a separate copy of the Movie.

Required private data members:

movie	Movie*	Pointer to the Movie being shown. The default value should be nullptr.
showTime	string	Time of the showing, such as 7:30 PM.
auditoriumNumber	int	Auditorium/screen number.
ticketPrice	float	Price of one ticket.
seatsAvailable	int	Number of seats currently available for the showing.

Required public member functions:

Function	Returns	Required behavior / purpose
Showing()	—	Default constructor. Initialize movie to nullptr, showTime to an empty string, and all numeric values to 0.
Showing(...)	—	Parameterized (overloaded) constructor. Accept a Movie pointer plus values for show time, auditorium number, ticket price, and available seats.
setMovie	void	Change the Movie pointer associated with this showing.
setShowTime	void	Change the show time.
setAuditoriumNumber	void	Change the auditorium number.
setTicketPrice	void	Change the ticket price.
setSeatsAvailable	void	Change the number of seats available.
getMovie	Movie*	Return the Movie pointer.
getShowTime	string	Return the show time.
getAuditoriumNumber	int	Return the auditorium number.
getTicketPrice	float	Return the ticket price.
getSeatsAvailable	int	Return the available-seat count.
editShowing	void	Display an edit menu that repeatedly allows the user to change show time, auditorium number, ticket price, or seats available until Done Editing is selected. The Movie association is not edited here.
printShowing	void	Print the associated movie title, show time, auditorium number, ticket price with two decimal places, and seats available. If movie is nullptr, print [NO MOVIE ASSIGNED].

Theater Class

The Theater object coordinates the entire program. It owns the dynamically allocated Movie and Showing objects and stores pointers to those objects in two fixed-capacity dynamic arrays.

Required private data members:

theaterName	string	Name of the theater.
movieArray	Movie**	Pointer to a dynamically allocated array of Movie pointers.
numMovies	int	Number of Movie objects currently stored.
maxMovies	int	Maximum number of Movie pointers the movie array can hold.
showingArray	Showing**	Pointer to a dynamically allocated array of Showing pointers.
numShowings	int	Number of Showing objects currently stored.
maxShowings	int	Maximum number of Showing pointers the showing array can hold.

Required public member functions:

Function	Returns	Required behavior / purpose
Theater(...)	—	Constructor. Accept the theater name, maximum movie capacity, and maximum showing capacity. Treat any capacity below 1 as 1. Dynamically allocate both pointer arrays and initialize every element to nullptr. Begin with 0 movies and 0 showings.
~Theater()	—	Destructor. Delete all dynamically allocated Showing objects and the showing array, then delete all dynamically allocated Movie objects and the movie array. Set released pointer variables to nullptr.
getNumMovies	int	Return the current number of movies.
getNumShowings	int	Return the current number of showings.
addMovie	void	If the movie array is full, print a capacity message and return. Otherwise, ask for title, release year, runtime, and rating; dynamically allocate a Movie; store its pointer in the next open element; increment numMovies; and print confirmation.
editMovie	void	If there are no movies, print a message and return. Otherwise, print the numbered movie names, validate the user's selection, and call editMovie() for the selected Movie.
addShowing	void	Require at least one Movie before a Showing can be created. Also prevent additions when the showing array is full. Print numbered movie names, validate the selected movie, ask for the remaining showing information, dynamically allocate a Showing that points to the selected Movie, store it, increment numShowings, and print confirmation.
editShowing	void	If there are no showings, print a message and return. Otherwise, print the numbered showing names, validate the user's selection, and call editShowing() for the selected Showing.
printMovies	void	If there are no movies, print a message. Otherwise, number the movies and call printMovie() for each one.

printShowings	void	If there are no showings, print a message. Otherwise, number the showings and call printShowing() for each one.
printMovieNames	void	Print a numbered list containing only the title of each current Movie. This is used when the user must select a Movie.
printShowingNames	void	Print a numbered list identifying each Showing by movie title and show time. This is used when the user must select a Showing.

Driver / main()

The Driver.cpp file is provided for you! Your program must work with the provided driver.

Input Validation Requirements

- The Movie edit menu must accept only 1–5.
- The Showing edit menu must accept only 1–5.
- When selecting a movie, accept only a number corresponding to a currently stored movie.
- When selecting a showing, accept only a number corresponding to a currently stored showing.
- When invalid input is entered, print an explanatory message and ask again rather than continuing with an invalid selection.

Required Testing Scenarios

Your program should be tested deliberately—not only with the “happy path.” The following scenarios correspond to the required behavior. Your wording and spacing may differ slightly from the sample output, but the behavior and stored data should be correct.

1. **Operations when the arrays are empty** — Before adding anything, attempt to edit a movie, add a showing, edit a showing, print movies, and print showings.
2. **Add and print movies** — Add multiple movies and verify every field prints correctly.
3. **Movie capacity reached** — Fill the Movie array to maxMovies and then attempt to add one more Movie.
4. **Edit a movie** — Choose a Movie by number, reject an invalid selection, edit at least two fields, finish editing, and print the movies again to verify the changes.
5. **Add and print showings** — Create multiple showings. Include at least two Showings that point to the same Movie and at least one Showing that points to a different Movie.
6. **Invalid movie selection while adding a showing** — When choosing the Movie for a Showing, enter an invalid movie number first, then a valid one.
7. **Showing capacity reached** — Fill the Showing array to maxShowings and then attempt to add one more Showing.
8. **Edit a showing** — Choose a Showing by number, reject an invalid selection, edit at least two fields, finish editing, and print showings to verify the changes.
9. **Quit / destructor behavior** — Quit normally and verify the program reaches the closing message and releases dynamically allocated memory without crashing.

Program 1 Report

Fill out this program report and take a screenshot of the confirmation page after submitting and name it **prog1report**. This is similar to the lab reports but for programming assignments instead.

Program Report Link: https://tntech.co1.qualtrics.com/jfe/form/SV_3vXGh5sYqieNStM

Sample Output

The examples below demonstrate the major test scenarios. Text highlighted in yellow represents user input. These are examples of the expected behavior; students may use different movie titles, theater names, times, and numeric values when testing.

Scenario A: Start-up, invalid menu selection, and adding movies

```
Welcome to the Movie Theater Management System!

What is the name of your theater? Crockett Midnight Cinema
Maximum number of movies: 3
Maximum number of showings: 4

What would you like to do?
1. Add One Movie
2. Edit One Movie
3. Add One Showing
4. Edit One Showing
5. Print All Movies
6. Print All Showings
7. Quit
```

```
CHOOSE 1-7: 9
Oops! You didn't enter a number in the valid range.
Enter 1-7: 1
Movie title: Hereditary
Release year: 2018
Runtime in minutes: 127
Rating: R
Hereditary was added.
```

```
CHOOSE 1-7: 1
Movie title: Poltergeist
Release year: 1982
Runtime in minutes: 114
Rating: PG
Poltergeist was added.
```

```
CHOOSE 1-7: 1
Movie title: Other Mommy
Release year: 2020
Runtime in minutes: 120
Rating: R
Other Mommy was added.
```

Scenario B: Movie capacity and printing all movies

For readability, repeated copies of the full main menu are omitted from this sample-output snippet.

```
CHOOSE 1-7: 1
Sorry, no more movies can be added. You have hit max capacity.

CHOOSE 1-7: 5
===== MOVIES =====

Movie #1
Title: Hereditary
Release Year: 2018
Runtime: 127 minutes
Rating: R

Movie #2
Title: Poltergeist
Release Year: 1982
Runtime: 114 minutes
Rating: PG

Movie #3
Title: Other Mommy
Release Year: 2020
Runtime: 120 minutes
Rating: R
```

Sample Output — Empty-Array Cases

Scenario C: Required behavior before anything has been added

For readability, repeated copies of the full main menu are omitted from this sample-output snippet.

```
CHOOSE 1-7: 2
There are no movies to edit.

CHOOSE 1-7: 3
You must have at least one movie before creating a showing.

CHOOSE 1-7: 4
There are no showings to edit.

CHOOSE 1-7: 5
There are no movies.

CHOOSE 1-7: 6
There are no showings.
```

Sample Output — Editing a Movie

Scenario D: Invalid Movie selection, then editing multiple fields

```
1. Hereditary
2. Poltergeist
3. Other Mommy
Which movie would you like to edit? 0
Oops! You didn't enter a valid movie number.
Which movie would you like to edit? 2

What would you like to edit?
1. Title
2. Release Year
3. Runtime
4. Rating
5. Done Editing
CHOOSE 1-5: 1
Current title: Poltergeist
New title: Poltergeist (1982)

CHOOSE 1-5: 4
Current rating: PG
New rating: PG-13

CHOOSE 1-5: 5
```

Sample Output — Adding Showings

Scenario E: Invalid Movie choice and multiple Showings pointing to Movie objects

```
CHOOSE 1-7: 3
1. Hereditary
2. Poltergeist (1982)
3. Other Mommy
Which movie is being shown? 9
Oops! You didn't enter a valid movie number.
Which movie is being shown? 1
Show time (example: 7:30 PM): 10:45 PM
Auditorium number: 2
Ticket price: $12.99
Seats available: 100
Showing was added.

CHOOSE 1-7: 3
1. Hereditary
2. Poltergeist (1982)
3. Other Mommy
Which movie is being shown? 1
Show time (example: 7:30 PM): 7:15 PM
Auditorium number: 3
Ticket price: $12.99
Seats available: 100
Showing was added.

CHOOSE 1-7: 3
1. Hereditary
2. Poltergeist (1982)
3. Other Mommy
Which movie is being shown? 3
Show time (example: 7:30 PM): 3:45 PM
Auditorium number: 1
Ticket price: $10.99
Seats available: 50
Showing was added.
```

Notice that the first two Showing objects both point to the same Hereditary Movie object. Your program should not create duplicate Movie objects for those showings.

Sample Output — Printing and Capacity for Showings

Scenario F: Print all Showings and reach maximum Showing capacity

```
CHOOSE 1-7: 6
===== SHOWINGS =====

Showing #1
Movie: Hereditary
Show Time: 10:45 PM
Auditorium: 2
Ticket Price: $12.99
Seats Available: 100

Showing #2
Movie: Hereditary
Show Time: 7:15 PM
Auditorium: 3
Ticket Price: $12.99
Seats Available: 100

Showing #3
Movie: Other Mommy
Show Time: 3:45 PM
Auditorium: 1
Ticket Price: $10.99
Seats Available: 50

Showing #4
Movie: Other Mommy
Show Time: 11:15 PM
Auditorium: 2
Ticket Price: $10.99
Seats Available: 100

CHOOSE 1-7: 3
Sorry, no more showings can be added. You have hit max capacity.
```

Sample Output — Editing a Showing

Scenario G: Invalid Showing selection, then editing multiple fields

```
1. Hereditary at 10:45 PM
2. Hereditary at 7:15 PM
3. Other Mommy at 3:45 PM
4. Other Mommy at 11:15 PM
Which showing would you like to edit? 0
Oops! You didn't enter a valid showing number.
Which showing would you like to edit? 3

What would you like to edit?
  1. Show Time
  2. Auditorium Number
  3. Ticket Price
  4. Seats Available
  5. Done Editing
CHOOSE 1-5: 3
Current ticket price: $10.99
New ticket price: $11.99

CHOOSE 1-5: 4
Current seats available: 50
New seats available: 45

CHOOSE 1-5: 5
```

Sample Output — Verifying Edits and Quitting

Scenario H: Reprint the changed Showing and quit normally

```
CHOOSE 1-7: 6
===== SHOWINGS =====
```



```
Showing #3  
Movie: Other Mommy  
Show Time: 3:45 PM  
Auditorium: 1  
Ticket Price: $11.99  
Seats Available: 45
```

```
... other showings omitted here ...
```

```
CHOOSE 1-7: 7
```

```
Closing the theater program.
```

```
Releasing showings...  
Releasing movies...
```

The destructor messages are included in this assignment so you can see that cleanup occurs when the Theater object is destroyed. Your program should terminate without a crash or memory-management error.